Diabetes Prediction and Glucose Estimation

This notebook trains two Random Forest models using `diabetes.csv`:

1. A classifier to predict the diabetes `Outcome`.
2. A regressor to estimate `Glucose` from the other available features.

Before running: Upload `diabetes.csv` to the Colab session, or update the file path in the dataset-loading cell.

1. Import Libraries

```
import pandas as pd
import numpy as np
import joblib
import sklearn

from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.feature_selection import SelectKBest, f_classif, f_regression
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor

print("Scikit-learn version:", sklearn.__version__)
```

```
Scikit-learn version: 1.6.1
```

2. Load and Preprocess the Dataset

```
# Load dataset
df = pd.read_csv("/content/drive/MyDrive/Colab Notebooks/diabetes.csv")

# Remove duplicate rows
df = df.drop_duplicates().copy()

# Replace invalid zero measurements with missing values
zero_as_missing = [
    "Glucose",
    "BloodPressure",
    "SkinThickness",
    "Insulin",
    "BMI"
]

for col in zero_as_missing:
    df[col] = df[col].replace(0, np.nan)

print("Dataset shape:", df.shape)
display(df.head())
print('\nMissing values by column:')
print(df.isnull().sum())
```

```
Dataset shape: (768, 9)
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72.0	35	169.5	33.6	0.627	50	1
1	1	85	66.0	29	102.5	26.6	0.351	31	0
2	8	183	64.0	32	169.5	23.3	0.672	32	1
3	1	89	66.0	23	94.0	28.1	0.167	21	0
4	0	137	40.0	35	168.0	43.1	2.288	33	1

```
Missing values by column:
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
```

3. Train the Diabetes Outcome Classifier

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
# Classification: predict Outcome
classification_features = [col for col in df.columns if col != "Outcome"]

X_classification = df[classification_features]
y_classification = df["Outcome"]

classifier_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("selector", SelectKBest(score_func=f_classif, k="all")),
    ("model", RandomForestClassifier(
        n_estimators=300,
        max_depth=8,
        min_samples_leaf=3,
        class_weight="balanced",
        random_state=42,
        n_jobs=-1
    ))
])

classifier_pipeline.fit(X_classification, y_classification)
print("Diabetes classifier trained successfully.")
```

```
Diabetes classifier trained successfully.
```

4. Save the Classification Model

```
# Save classifier bundle for Flask
joblib.dump({
    "pipeline": classifier_pipeline,
    "features": classification_features
}, "diabetes_classifier.joblib")

print("Created: diabetes_classifier.joblib")
```

```
Created: diabetes_classifier.joblib
```

5. Train the Glucose Regression Model

```
# Regression: predict Glucose using other columns
regression_features = [
    col for col in df.columns
    if col not in ["Glucose", "Outcome"]
]

X_regression = df[regression_features]
y_regression = df["Glucose"]

# Rows with missing target values cannot be used for training
valid_rows = y_regression.notna()

regressor_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("selector", SelectKBest(score_func=f_regression, k="all")),
    ("model", RandomForestRegressor(
        n_estimators=300,
        max_depth=8,
        min_samples_leaf=3,
        random_state=42,
        n_jobs=-1
    ))
])

regressor_pipeline.fit(
    X_regression.loc[valid_rows],
    y_regression.loc[valid_rows]
)

print("Glucose regressor trained successfully.")
```

```
Glucose regressor trained successfully.
```

6. Save the Regression Model

```
# Save regressor bundle for Flask
joblib.dump({
    "pipeline": regressor_pipeline,
    "features": regression_features
}, "glucose_regressor.joblib")

print("Created: glucose_regressor.joblib")
```

```
Created: glucose_regressor.joblib
```

7. Summary

```
print("Both models trained and saved successfully!")
print("Created: diabetes_classifier.joblib")
print("Created: glucose_regressor.joblib")
```

```
Both models trained and saved successfully!
Created: diabetes_classifier.joblib
Created: glucose_regressor.joblib
```

Flask Code:

```
from flask import Flask, render_template, request
import pandas as pd
import numpy as np
import joblib

app = Flask(__name__)

# Load the trained models
classifier_bundle = joblib.load("diabetes_classifier.joblib")
regressor_bundle = joblib.load("glucose_regressor.joblib")

classifier = classifier_bundle["pipeline"]
regressor = regressor_bundle["pipeline"]

classification_features = classifier_bundle["features"]
regression_features = regressor_bundle["features"]


@app.route("/")
def home():
    return render_template("index.html")


@app.route("/predict", methods=["POST"])
def predict():
    try:
        # Get values submitted through the HTML form
        pregnancies = int(request.form["Pregnancies"])
        glucose = float(request.form["Glucose"])
        blood_pressure = float(request.form["BloodPressure"])
        skin_thickness = float(request.form["SkinThickness"])
        insulin = float(request.form["Insulin"])
        bmi = float(request.form["BMI"])
        dpf = float(request.form["DiabetesPedigreeFunction"])
        age = int(request.form["Age"])

        values = {
            "Pregnancies": pregnancies,
            "Glucose": glucose,
            "BloodPressure": blood_pressure,
            "SkinThickness": skin_thickness,
            "Insulin": insulin,
            "BMI": bmi,
            "DiabetesPedigreeFunction": dpf,
            "Age": age
        }

        # Reject invalid values before making predictions
```

```
        if any(not np.isfinite(float(v)) for v in values.values()):
            raise ValueError("Please enter valid numeric values.")

        if any(float(v) < 0 for v in values.values()):
            raise ValueError("Measurements cannot be negative.")

        # Build input rows in the exact feature order used in training
        classification_input = pd.DataFrame(
            [[values[col] for col in classification_features]],
            columns=classification_features
        )

        regression_input = pd.DataFrame(
            [[values[col] for col in regression_features]],
            columns=regression_features
        )

        # Apply the same missing-value assumption used during training
        zero_as_missing = [
            "Glucose", "BloodPressure",
            "SkinThickness", "Insulin", "BMI"
        ]

        for col in zero_as_missing:
            if col in classification_input.columns:
                classification_input.loc[
                    classification_input[col] == 0, col
                ] = np.nan

            if col in regression_input.columns:
                regression_input.loc[
                    regression_input[col] == 0, col
                ] = np.nan

        # Classification prediction
        outcome = int(classifier.predict(classification_input)[0])
        probability = float(
            classifier.predict_proba(classification_input)[0, 1]
        )

        # Regression prediction
        glucose_estimate = float(
            regressor.predict(regression_input)[0]
        )

        if outcome == 1:
            result = "Diabetes-positive class (1)"
        else:
            result = "Diabetes-negative class (0)"

        return render_template(
            "index.html",
            result=result,
            probability=f"{probability:.1%}",
            glucose_estimate=f"{glucose_estimate:.1f}",
            submitted=True
        )

    except (ValueError, KeyError, TypeError):
        return render_template(
            "index.html",
            error="Please enter valid numeric values in all fields.",
            submitted=False
        )


if __name__ == "__main__":
    app.run(debug=True)
```

HTML Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Diabetes Prediction System</title>
```

```
<style>
    *{
        box-sizing: border-box;
    }

    body {
        margin: 0;
        padding: 25px 15px;
        font-family: Arial, sans-serif;
        background: #f0f4f8;
        color: #243247;
    }

    .container {
        max-width: 650px;
        margin: auto;
        padding: 28px;
        background: white;
        border-radius: 14px;
        box-shadow: 0 5px 20px rgba(0, 0, 0, 0.08);
    }

    h1 {
        text-align: center;
        color: #155e75;
        margin-top: 0;
    }

    .subtitle {
        text-align: center;
        color: #64748b;
        margin-bottom: 25px;
    }

    .form-grid {
        display: grid;
        grid-template-columns: 1fr 1fr;
        gap: 16px;
    }

    label {
        display: block;
        font-weight: bold;
        margin-bottom: 7px;
    }

    input {
        width: 100%;
        padding: 11px;
        border: 1px solid #cbd5e1;
        border-radius: 7px;
        font-size: 15px;
    }

    button {
        width: 100%;
        padding: 13px;
        margin-top: 22px;
        border: none;
        border-radius: 7px;
        background: #0e7490;
        color: white;
        font-size: 16px;
        font-weight: bold;
        cursor: pointer;
    }

    button:hover {
        background: #155e75;
    }

    .result {
        margin-top: 24px;
        padding: 18px;
        border-radius: 8px;
        background: #ecfeff;
        border: 1px solid #a5f3fc;
        overflow-wrap: anywhere;
    }

    .error {
```

```
            margin-top: 20px;
            padding: 12px;
            color: #991b1b;
            background: #fee2e2;
            border-radius: 7px;
        }

        .note {
            margin-top: 22px;
            font-size: 12px;
            color: #64748b;
            line-height: 1.5;
        }

        @media (max-width: 500px) {
            .form-grid {
                grid-template-columns: 1fr;
            }

            .container {
                padding: 20px;
            }
        }
    </style>
</head>

<body>
    <div class="container">

        <h1>Diabetes Prediction</h1>

        <p class="subtitle">
            Decision Tree and Random Forest Machine Learning Project
        </p>

        <form action="/predict" method="POST">
            <div class="form-grid">

                <div>
                    <label for="Pregnancies">Pregnancies</label>
                    <input type="number"
                           id="Pregnancies"
                           name="Pregnancies"
                           min="0" max="20"
                           value="1" required>
                </div>

                <div>
                    <label for="Glucose">Glucose</label>
                    <input type="number"
                           id="Glucose"
                           name="Glucose"
                           min="0" max="300"
                           step="any" value="120" required>
                </div>

                <div>
                    <label for="BloodPressure">Blood Pressure</label>
                    <input type="number"
                           id="BloodPressure"
                           name="BloodPressure"
                           min="0" max="200"
                           step="any" value="70" required>
                </div>

                <div>
                    <label for="SkinThickness">Skin Thickness</label>
                    <input type="number"
                           id="SkinThickness"
                           name="SkinThickness"
                           min="0" max="120"
                           step="any" value="25" required>
                </div>

                <div>
                    <label for="Insulin">Insulin</label>
                    <input type="number"
                           id="Insulin"
                           name="Insulin"
                           min="0" max="1000"
                           step="any" value="80" required>
                </div>
```

```
                <div>
                    <label for="BMI">BMI</label>
                    <input type="number"
                           id="BMI"
                           name="BMI"
                           min="0" max="100"
                           step="any" value="28" required>
                </div>

                <div>
                    <label for="DiabetesPedigreeFunction">
                        Diabetes Pedigree Function
                    </label>
                    <input type="number"
                           id="DiabetesPedigreeFunction"
                           name="DiabetesPedigreeFunction"
                           min="0" max="3"
                           step="any" value="0.4" required>
                </div>

                <div>
                    <label for="Age">Age</label>
                    <input type="number"
                           id="Age"
                           name="Age"
                           min="1" max="120"
                           value="30" required>
                </div>

            </div>

            <button type="submit">
                Generate Predictions
            </button>
        </form>

        {% if submitted %}
        <div class="result">
            <h2>Prediction Results</h2>

            <p>
                <strong>Classification:</strong>
                {{ result }}
            </p>

            <p>
                <strong>Model-estimated probability of class 1:</strong>
                {{ probability }}
            </p>

            <p>
                <strong>Estimated glucose level:</strong>
                {{ glucose_estimate }} mg/dL
            </p>
        </div>
        {% endif %}

        {% if error %}
        <div class="error">
            {{ error }}
        </div>
        {% endif %}

        <p class="note">
            Educational demonstration only. This application is not a
            medical diagnostic tool. Predictions must not be used to make
            health decisions. The displayed probability is a model output,
            not a clinically validated personal risk estimate.
        </p>

    </div>
</body>
</html>
```